

Urban Data Integration Platform

Week 2 Design Report — Analytics, Data Products, and Optimisation

Module: Data-intensive Computing **Project:** NYC Taxi Urban Data Platform

1. Purpose and analytical requirements

Week 2 turns the canonical Week 1 integrated Delta table into a reusable analytical layer. The source, **data/gold/integrated_taxi_trips**, contains 8,480,836 cleaned and enriched trip records. Week 2 is read-only with respect to that table and answers six stakeholder questions:

ID	Analytical question
Q1	Which pickup zones have the highest monthly demand?
Q2	How do trip distances vary with weather conditions?
Q3	Does PM2.5 air quality correspond with taxi demand?
Q4	Which zones have the greatest demand variability under different weather conditions?
Q5	What are the busiest pickup hours for each day of the week?
Q6	How does taxi demand change month over month?

2. Analytical query design

The six analyses are reusable Spark SQL DataFrame functions in **src/analytics/queries.py**. Q1 groups by month and pickup zone; Q2 groups distance statistics by normalised weather condition; Q3 aggregates demand by PM2.5 category; Q4 performs a zone/condition aggregation followed by standard deviation; Q5 derives weekday and hour from local pickup timestamps; and Q6 calculates monthly totals and month-over-month change. The 265-row canonical Silver taxi-zone Delta lookup supplies zone names, avoiding a parallel raw-CSV dependency.

All timestamps use the Week 1 **America/New_York** Spark session timezone. This makes day-of-week and hour results meaningful for NYC operations rather than dependent on the machine timezone.

3. Reusable data products

Four automatically generated Gold-layer Delta products are created by `src/analytics/data_products.py`. Every product records `data_source`, `creation_time`, `refresh_time`, and `schema_version` metadata. The generator overwrites only these derived tables; it does not change Week 1 Bronze, Silver, or integrated Gold data.

Product	Primary users	Decision value
daily_mobility	Transport planners	Daily zone demand, distance, revenue, and fare trends without a trip-level scan.
taxi_zone_statistics	Operations analysts	Whole-period comparison of zones for demand, distance, revenue, and fare.
weather_impact	Mobility/weather analysts	Trip outcomes grouped by temperature bucket and weather condition.
air_quality_impact	Policy analysts	Trip outcomes grouped by PM2.5 air-quality category.

Storage and refresh trade-off

The four products occupy **1.25 MB** against a **1.19 GB** integrated source: a 0.10% product/source ratio. This negligible storage overhead is justified by fast access to repeated aggregate questions. Batch overwrite is suitable for the fixed Week 2 dataset; a continuously arriving multi-city feed should instead use incremental Delta MERGE refreshes.

4. Optimisation strategy

The optimisation module evaluates caching for repeated aggregation, partition filtering, broadcast joins, and Adaptive Query Execution (AQE). Each experiment checks that baseline and optimised outputs are identical and prints EXPLAIN FORMATTED output as physical-plan evidence. Timings are the median of warm runs 2–3; run 1 is excluded because Spark and Delta cold-start work would distort a steady-state comparison.

5. Measured optimisation results

Local run context: Spark 3.5.9, Delta Lake 3.2.1, 8,480,836 integrated trips, and a 265-row taxi-zone lookup.

Technique	Baseline	Optimised	Change	Interpretation
Cache repeated aggregation	1.070 s	0.536 s	49.91% faster	Persisted input avoids repeat read and recomputation.
year = 2024 filter	0.861 s	0.778 s	9.64% faster	Plan shows PartitionFilters; most data is already in one year.
Broadcast taxi-zone lookup	3.368 s	1.244 s	63.06% faster	Avoids shuffling 8.48M trips for a 265-row lookup.
AQE	1.187 s	1.232 s	3.79% slower	Already-broadcastable local join leaves little runtime adaptation.

Interpretation

Broadcast join is the strongest result because the dimension/source size asymmetry is extreme. AQE is not universally harmful: for this small, already well-planned local workload, its planning overhead exceeds its benefit. It should be re-evaluated with production-size partitions and skewed keys. Partition pruning helps, but its benefit is limited because the available data is primarily one year.

Q1 and Q4 remain the most demanding analyses. Both scan the full trip table and aggregate many records; Q4 adds a second shuffle for standard deviation across weather-condition groups. Caching improves repeated access but cannot remove the first-run scan and aggregation cost.

6. Engineering decisions, scaling, and reproducibility

Shared paths and weather mappings are centralised in `src/analytics/constants.py`, preventing query, product, and optimisation code from drifting apart. `src/analytics/runtime.py` sets a process-local writable Spark temporary directory before startup. On Windows this avoids a non-fatal cleanup warning caused by inheriting `C:\Windows\Temp`, without changing data locations or query output.

Scaling to ten cities

Recommendation	Reason
Partition by city	Allows city filters to skip unrelated files.
Review shuffle partitions	Eight local partitions will under-parallelise a much larger distributed workload.
Cluster/Z-order frequent filters	Improves file locality for city, zone, year, and month predicates.
Use incremental Delta MERGE refresh	Avoids costly full product overwrites as new trips arrive.
Run on a real cluster	Local mode is constrained by one machine's CPU, memory, and disk.

Reproduce the evidence

1. Run the Week 1 ingestion and integration pipeline. 2. Run `python -m src.analytics.data_products`. 3. Run `python -m src.analytics.optimization --evaluate`. 4. Run `python run_week2_query_benchmark.py` to time all six analytical functions three times and report their warm-run median. The repository README gives the full fresh-clone setup.

Submission contents: source code, analytical notebook, four task reports, this 4-page design report, benchmark methodology/results, and executable setup instructions. Week 1 implementation is not modified by the Week 2 analytics work.